



# FRAUD DETECTION

## WITH MACHINE LEARNING

Internship Project  
Batuhan Meydanlı  
August/2025

# CONTENTS

|                                  |     |
|----------------------------------|-----|
| Project Objective                | 3   |
| Algorithms Used                  | 4   |
| Isolation Forest & One Class SVM | 5–6 |
| Feature Engineering              | 7   |
| Methods Used                     | 8   |
| Z-Score                          | 9   |
| Dynamic Thresholding             | 10  |
| Sample Outputs & Visualizations  | 11  |
| Hyperparameter Optimization      | 12  |
| Libraries Used                   | 13  |

# PROJECT OBJECTIVE

Many users are exposed to fraud attempts during everyday financial transactions. These incidents may lead not only to individual financial losses, but also to reputational and operational security risks for service providers.

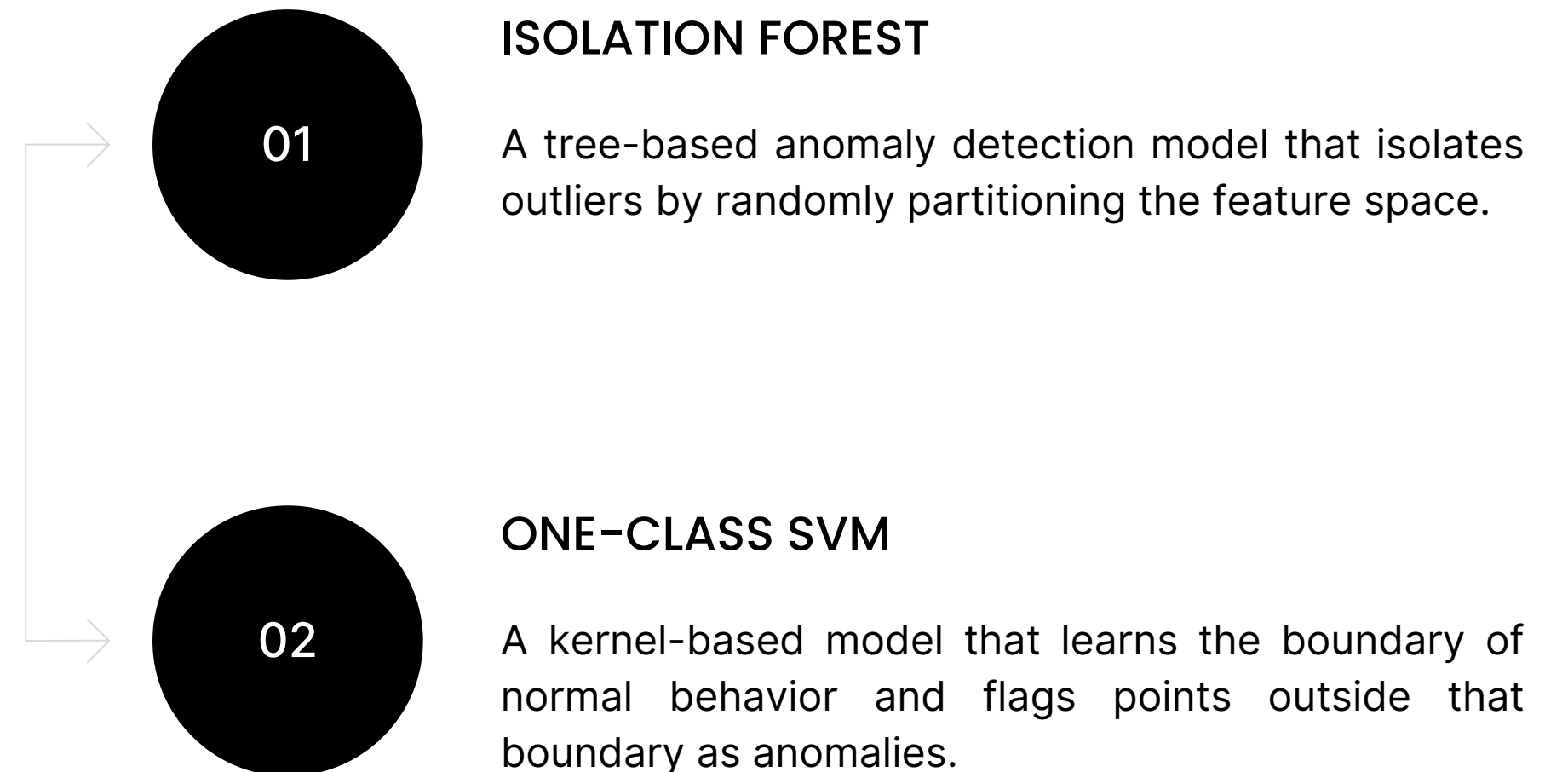
This project developed a data-driven and statistically grounded anomaly detection approach to address the limitations of rule-based systems. The objective was to model users' transaction habits, learn normal behavioral patterns, and detect deviations from those patterns.

Both traditional metric-based analysis methods, such as Z-Score and Dynamic Thresholding, and machine learning-based models, including Isolation Forest and One-Class SVM, were evaluated together. The results highlighted the value of adaptive learning-based approaches in flexibility, detection quality, and adaptation to emerging fraud scenarios.



# ALGORITHMS USED

The two main models used in this project follow an unsupervised learning approach. They do not require labeled fraud samples; instead, they analyze the natural structure of the data and focus on detecting unusual behavior. This makes them suitable for domains where behavioral patterns change over time and manual labeling is difficult.



# ISOLATION FOREST

Isolation Forest is a decision tree-based anomaly detection algorithm. Its core approach is based on the idea of isolating data points rather than modeling statistical distributions. The model assumes that anomalous observations are inherently easier to separate, or isolate, from the rest of the data.

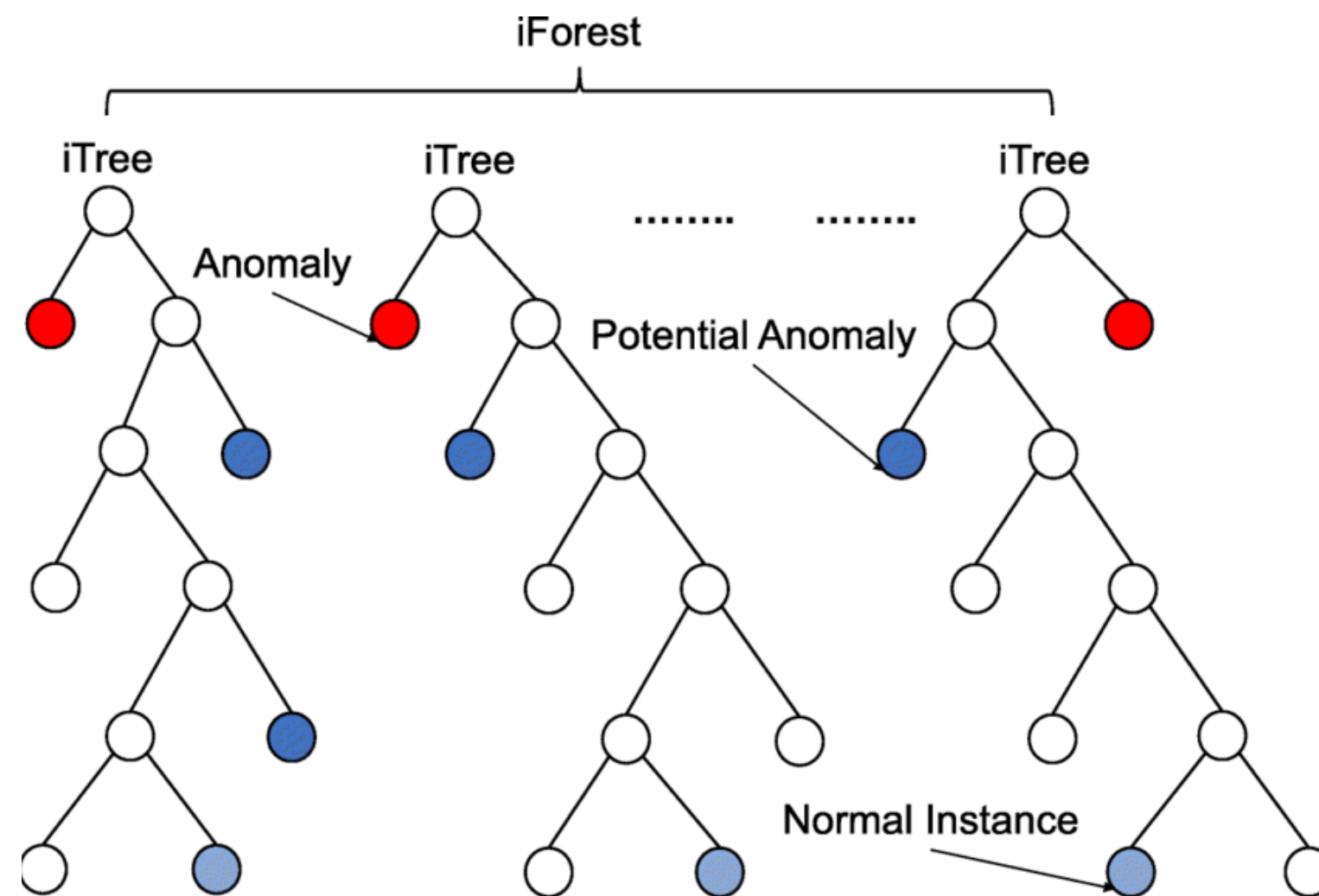
At each step, the algorithm randomly selects a feature and a threshold value for that feature, then recursively partitions the data into binary decision trees. Each tree, referred to as an iTree, represents the number of splits required to isolate an observation. The average isolation depth is calculated for each data point, and observations that can be isolated with fewer splits are identified as anomalies.

- Anomalous observations can typically be isolated at higher levels of the tree with fewer splits.
- Normal observations generally require more splits and are isolated at deeper levels of the tree.

The main hyperparameters used in the project are:

- `n_estimators`: Total number of trees to be generated
- `max_samples`: Number of samples used to build each tree
- `max_features`: Number of features considered during each split
- `contamination`: Assumed proportion of anomalies in the dataset
- `bootstrap`: Determines whether sampling with replacement is used
- `random_state`: Ensures reproducibility of results (42)

Isolation Forest performs effectively on high-dimensional, unlabeled datasets and provides a flexible and scalable anomaly detection approach without requiring rule-based models.



*How Isolation Forest Works*

# ONE-CLASS SVM

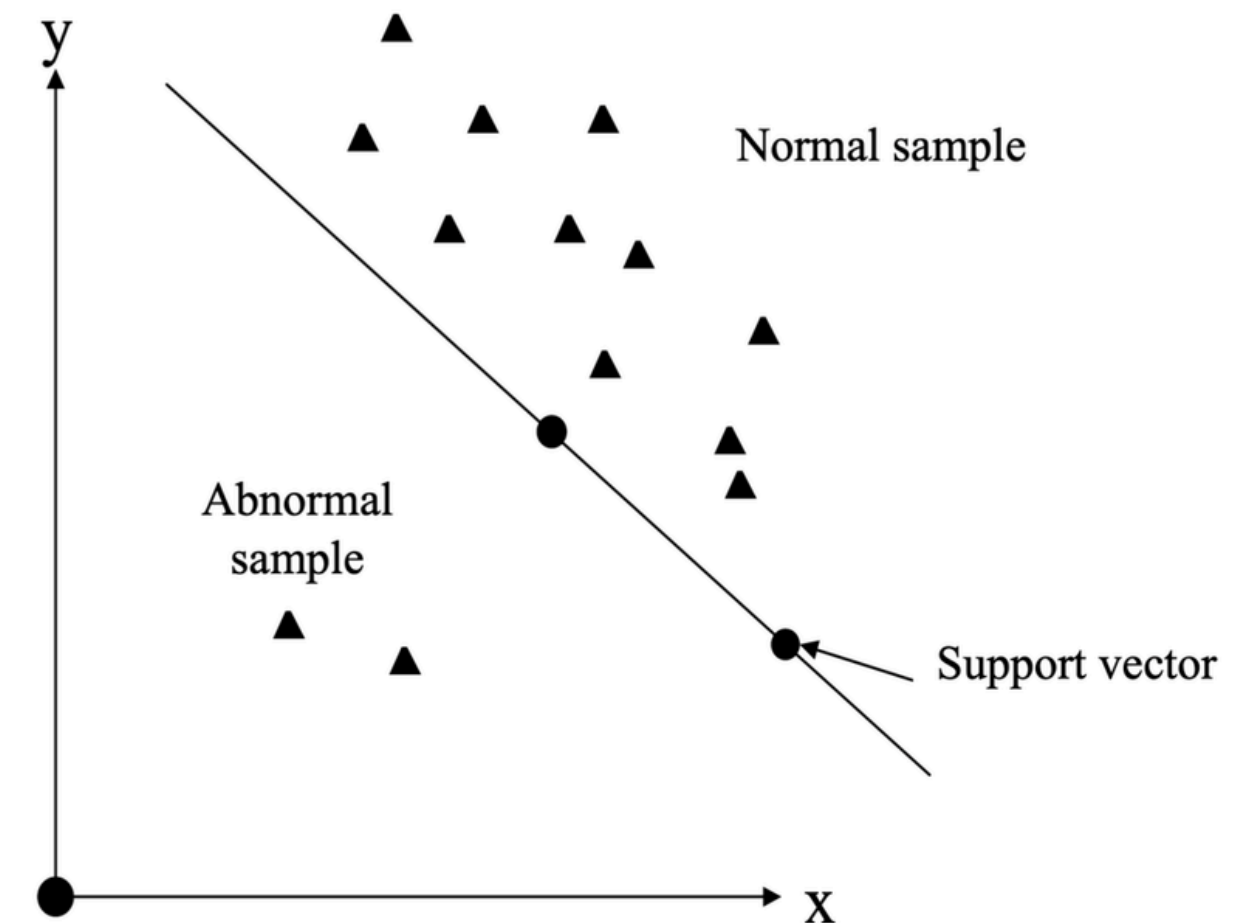
One-Class SVM is an unsupervised machine learning algorithm particularly suited to scenarios where labeled anomaly data is unavailable. Its primary objective is to learn a decision boundary around data representing “normal” behavior and identify observations that fall outside this boundary as potential anomalies.

Unlike conventional classification algorithms, One-Class SVM learns from a single class and therefore does not distinguish between multiple labeled categories in the traditional sense. Instead, it focuses on modeling the distribution of observations considered normal. In contrast to standard Support Vector Machine (SVM) approaches, it learns a decision surface that encloses the region occupied by normal data. Observations that fall outside this surface are treated as deviations from the expected data distribution and are classified as potential anomalies.

The following hyperparameters were considered in the One-Class SVM model used in the project:

- kernel: Kernel function used to transform the feature space, typically RBF (Radial Basis Function)
- nu: Controls the model's tolerance and acts as an upper bound on the proportion of anomalies
- gamma: Determines the influence of the RBF kernel and affects the complexity of the decision boundary

One-Class SVM can provide effective results for datasets with complex decision boundaries. However, it is sensitive to feature scaling, and careful hyperparameter selection is required.



Isolation Forest Flag dağılımı:

isoforest\_flag

0 140491

1 1420

Name: count, dtype: int64

One-Class SVM Flag dağılımı:

ocsvm\_flag

0 140509

1 1402

Name: count, dtype: int64

# FEATURE ENGINEERING

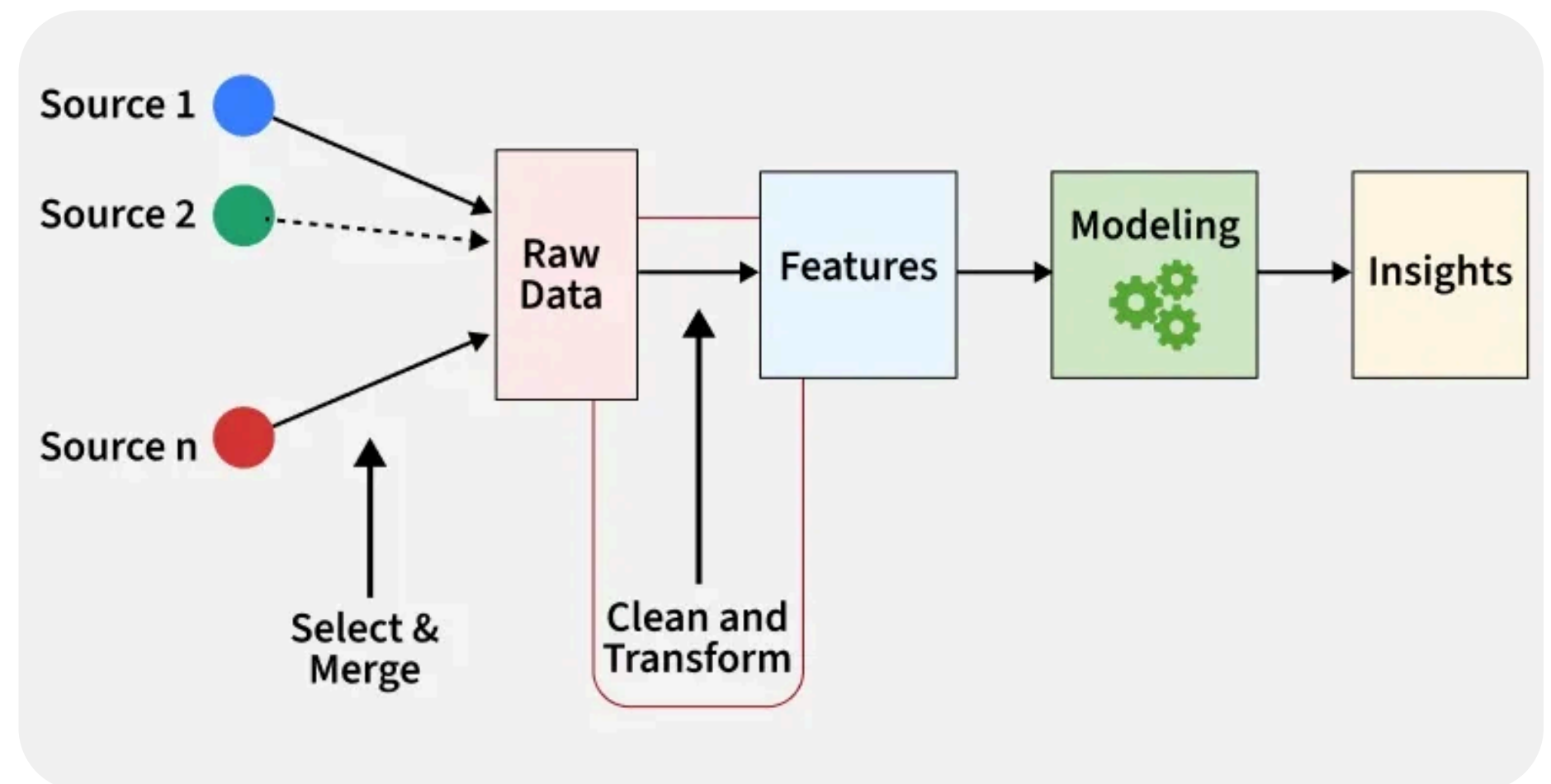
Feature engineering is one of the key factors that directly influences model performance in machine learning projects, as the quality of the features used for training has a significant impact on the model's ability to identify meaningful patterns. In this project, a comprehensive feature engineering process was carried out based on the assumption that anomalies could be identified not only through transaction amounts, but also through contextual and behavioral patterns.

For financial transaction data, time-series-based, statistical, and contextual features were derived to capture each user's transaction habits and deviations from their typical behavior. This enabled the model to detect not only isolated abnormal transactions, but also broader deviations from an individual user's overall behavioral profile.

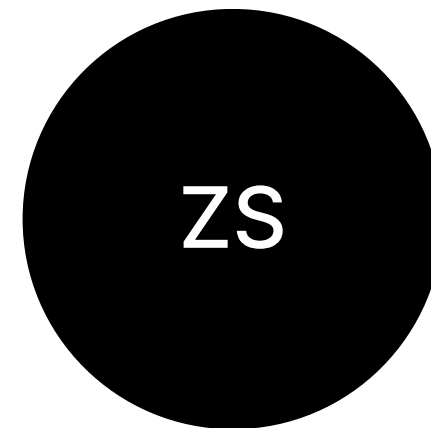
Some of the engineered features included:

- Average transaction amount over the last five transactions
- Transaction-level and daily Z-score values
- Transaction frequency within predefined time windows (10 minutes and 1 hour)
- Repeated transactions with identical amounts
- Use of new IP addresses and overall IP diversity
- Number of consecutive declined transactions
- Time elapsed since the previous transaction and transaction velocity
- Daily currency diversity and currency-change flags
- Deviation from typical transaction hours
- Number of different users associated with the same card and card-sharing intensity

Each feature was designed to characterize users' typical behavioral patterns and help identify meaningful deviations from those patterns. Many of these features transformed contextual relationships within transaction histories into numerical representations, improving the effectiveness of the unsupervised anomaly detection models.

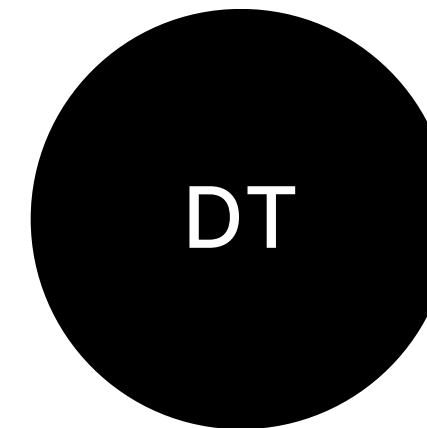


# METHODS USED WITH FEATURE ENGINEERING



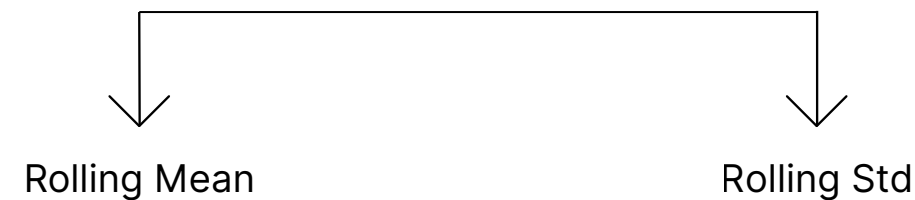
## Z-SCORE

Measures how much a transaction deviates from normal behavior based on the mean and standard deviation within a selected time window.



## DYNAMIC THRESHOLDING

Defines flexible lower and upper anomaly thresholds that are recalculated for each window according to the user's transaction behavior



A Rolling Mean calculates the average of values within a defined window to capture the underlying trend over time. Rolling Standard Deviation (Rolling Std) measures the degree of variability within the same window. Together, these rolling statistics form the basis of the Z-Score and Dynamic Thresholding methods.

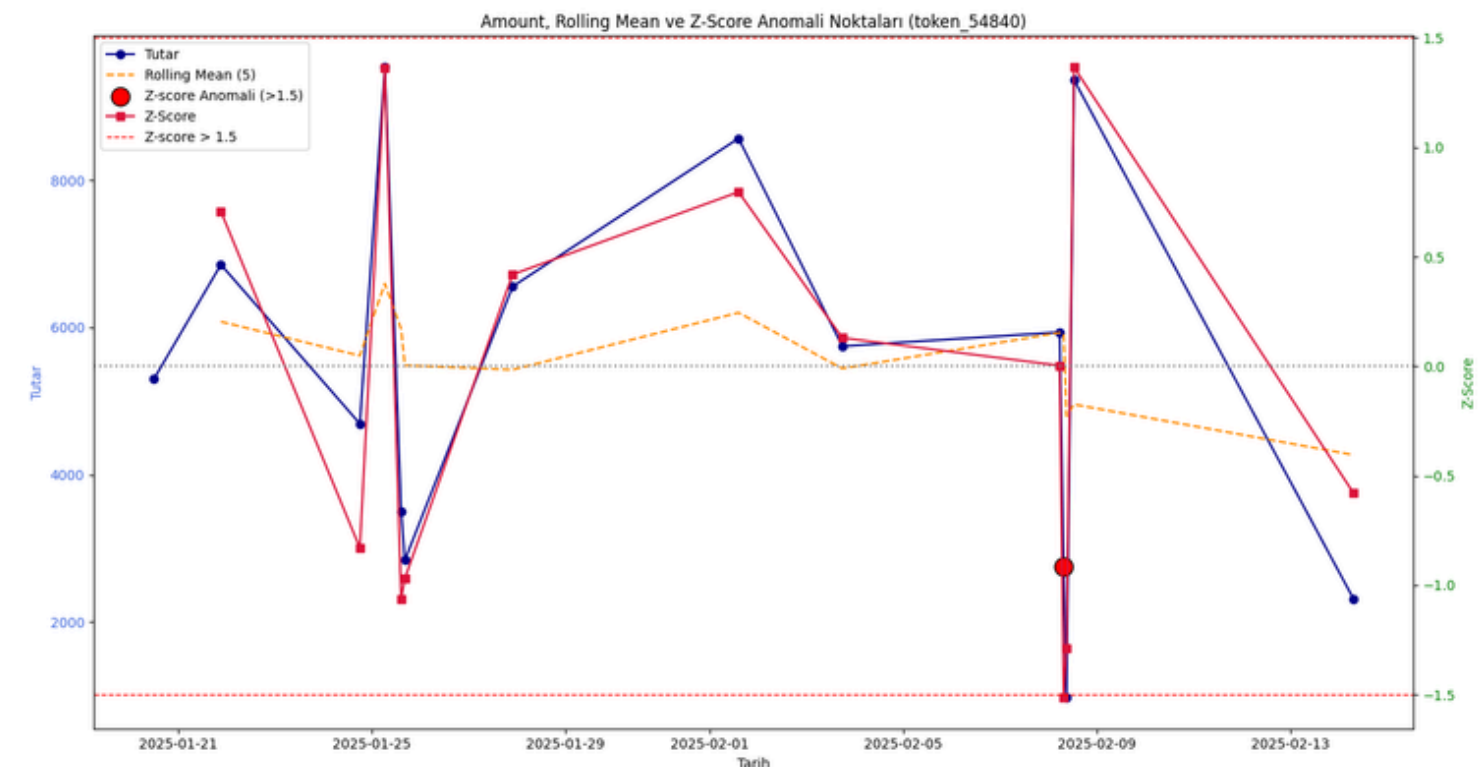
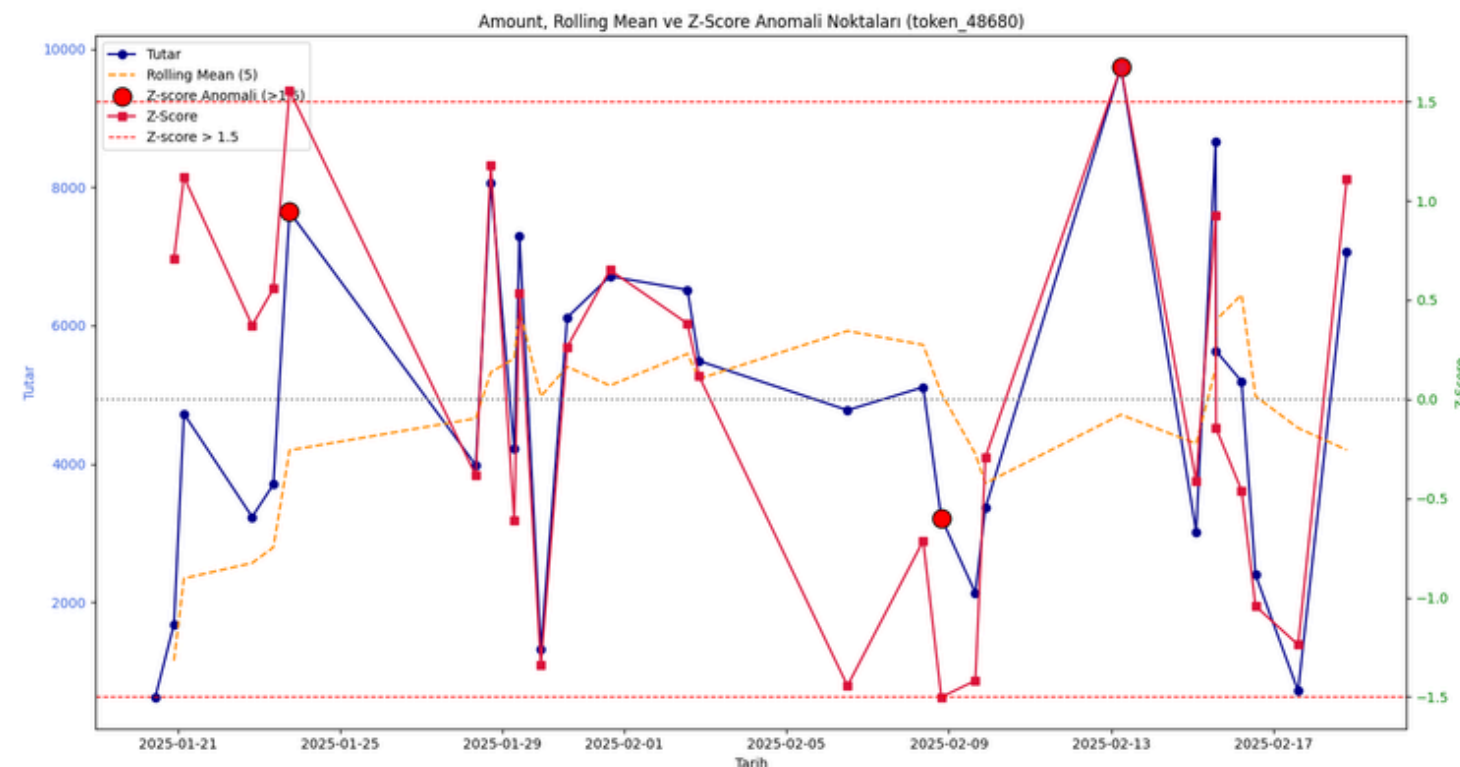
# Z-SCORE

Z-Score is a standard statistical method used to measure how far a value deviates from the expected behavior within its own distribution. In financial transaction data, the mean and standard deviation of transactions within a defined time window are calculated to obtain a Z-score for each transaction. This score provides a numerical indication of how strongly a transaction behaves as an outlier.

The Z-Score calculation used in the model was applied as follows:

- Rolling Window: A defined window is selected from the user's historical transactions, such as the most recent 10 transactions.
- Mean and Standard Deviation: The mean and standard deviation within this window are used to calculate the Z-score of the new transaction.
- Threshold: A fixed threshold of  $|z| > 1.5$  is applied, and transactions exceeding this value are flagged as anomalies.

Because the Z-Score method relies on a fixed threshold, it does not directly account for periodic changes over time. However, it is effective at identifying statistically significant deviations within a defined transaction window.



The visuals illustrate two different examples, showing transaction amount, rolling mean, and Z-Score values, with detected anomaly points highlighted for each case.

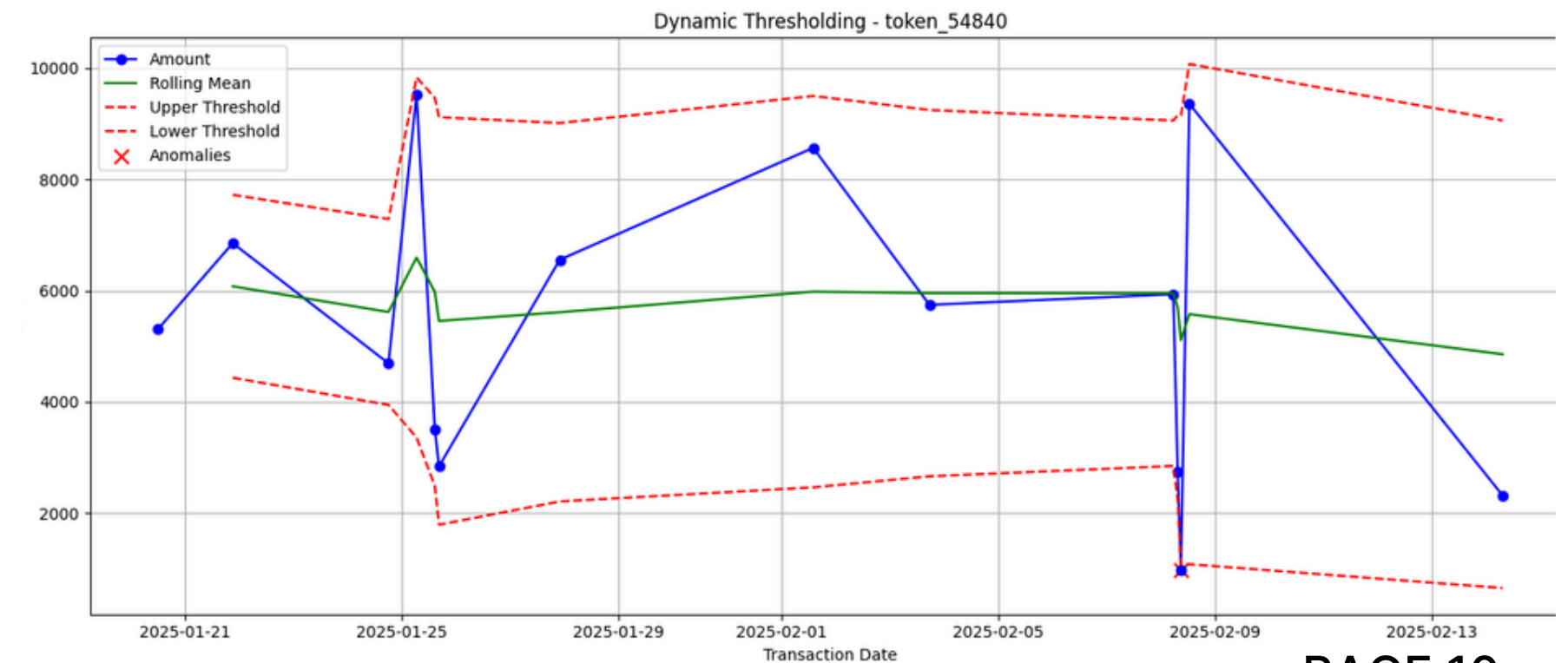
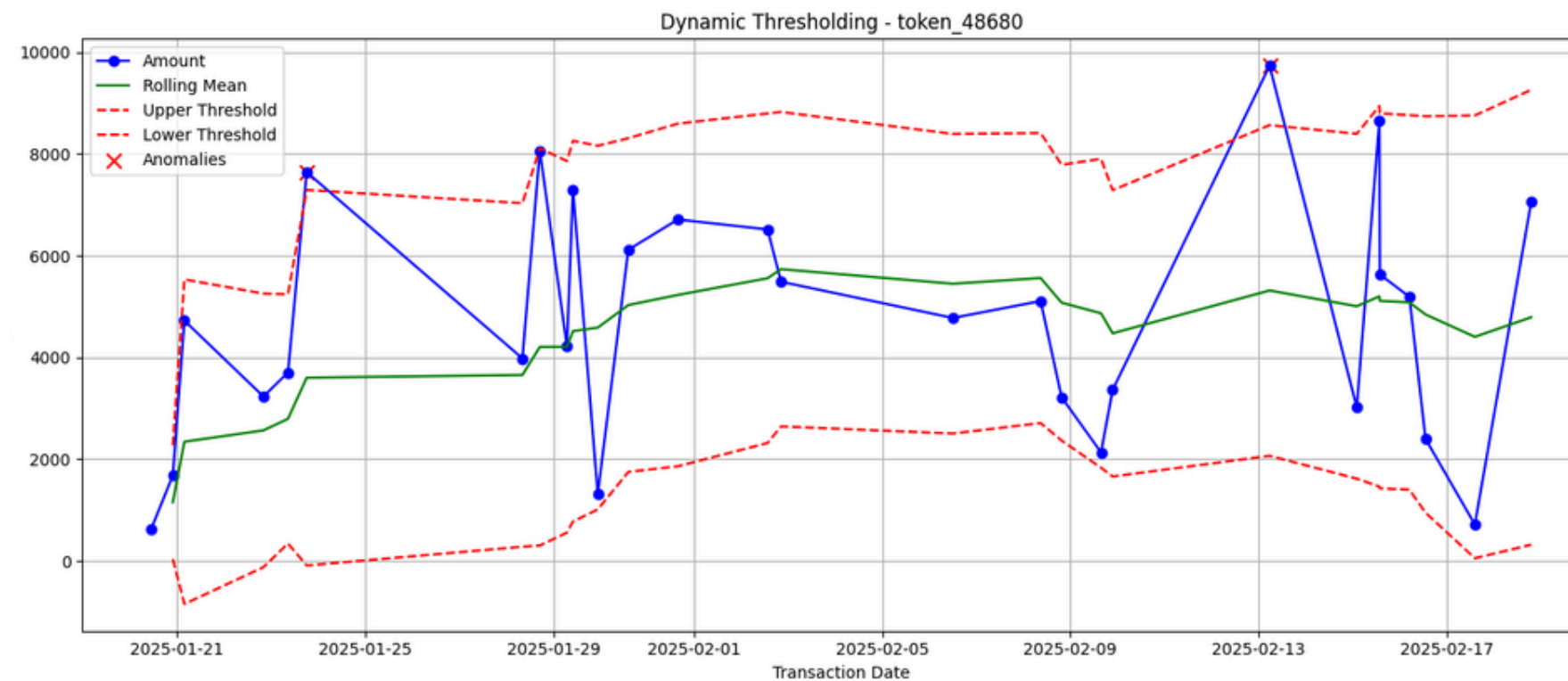
# DYNAMIC THRESHOLDING

Dynamic Thresholding is a flexible, context-aware anomaly detection approach used in financial transaction analysis when fixed thresholds are insufficient. Instead of evaluating each transaction against a static value, the method assesses it within the context of the user's own transaction history. This allows the detection process to adapt to user-specific transaction patterns and provides a more responsive view of anomalous behavior.

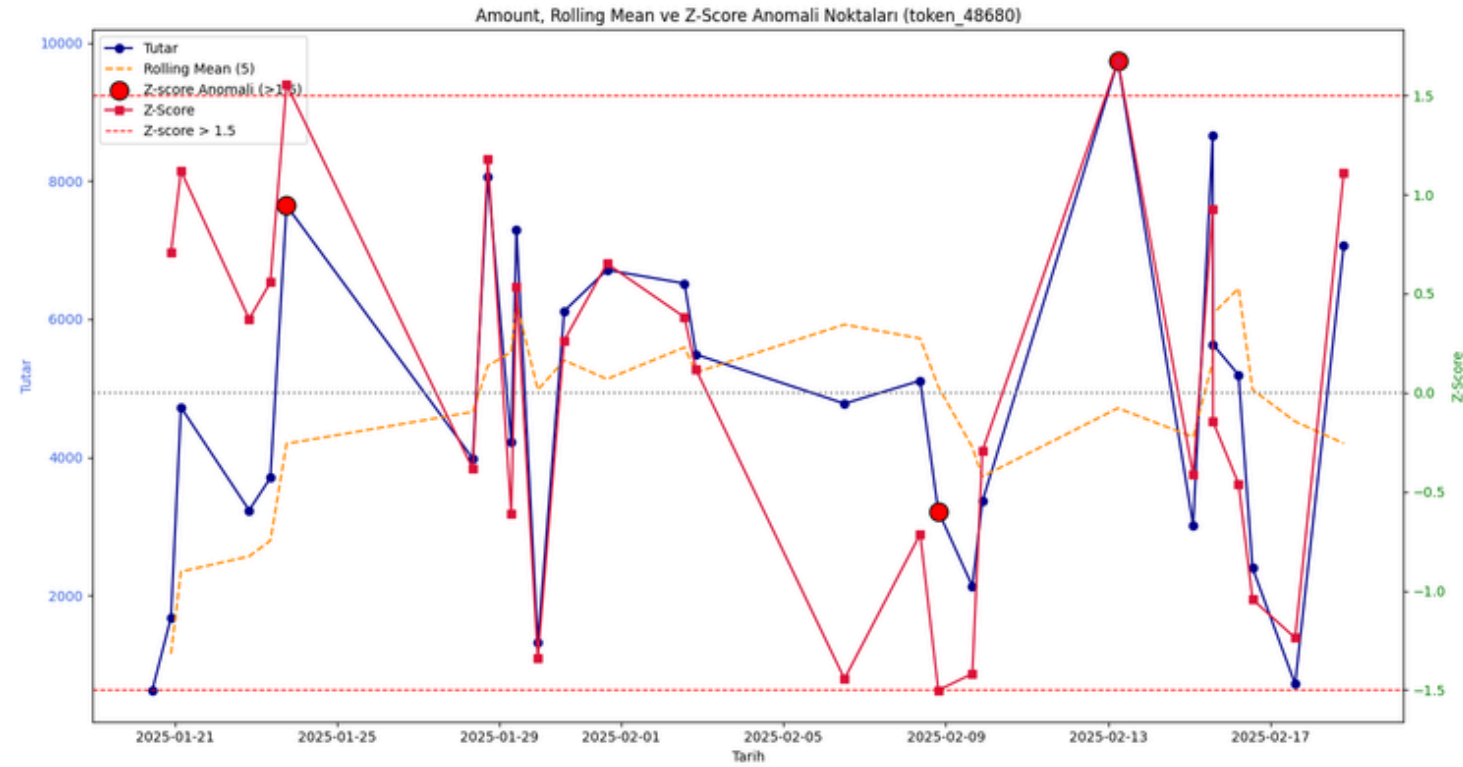
The Dynamic Thresholding process used in the model was applied as follows:

- Rolling Mean: The average of historical transactions within a defined window is calculated. This value represents the typical transaction level for that period.
- Rolling Standard Deviation (Rolling Std): The variability of transaction values within the same window is measured.
  - Dynamic Threshold Calculation: Upper threshold:  $\text{Mean} + k \times \text{Standard Deviation}$
  - Lower threshold:  $\text{Mean} - k \times \text{Standard Deviation}$
  - Each transaction is compared against these dynamically calculated thresholds, and transactions falling outside the range are flagged as potential anomalies.

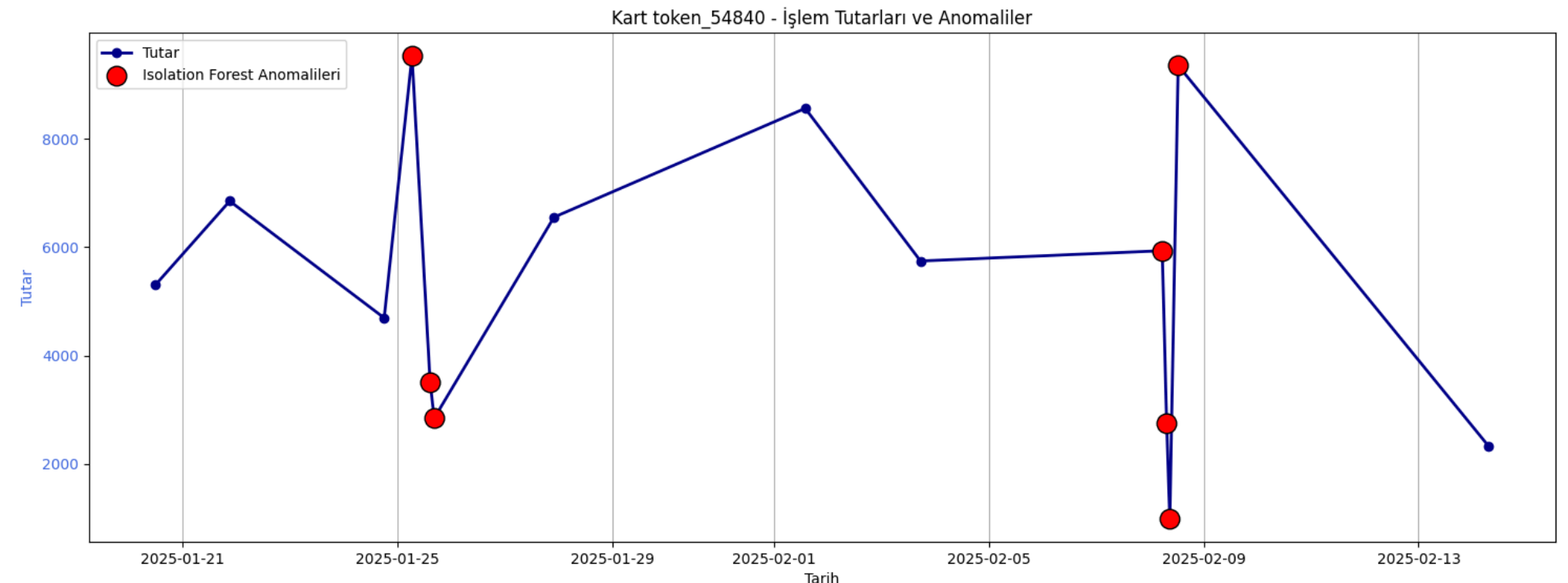
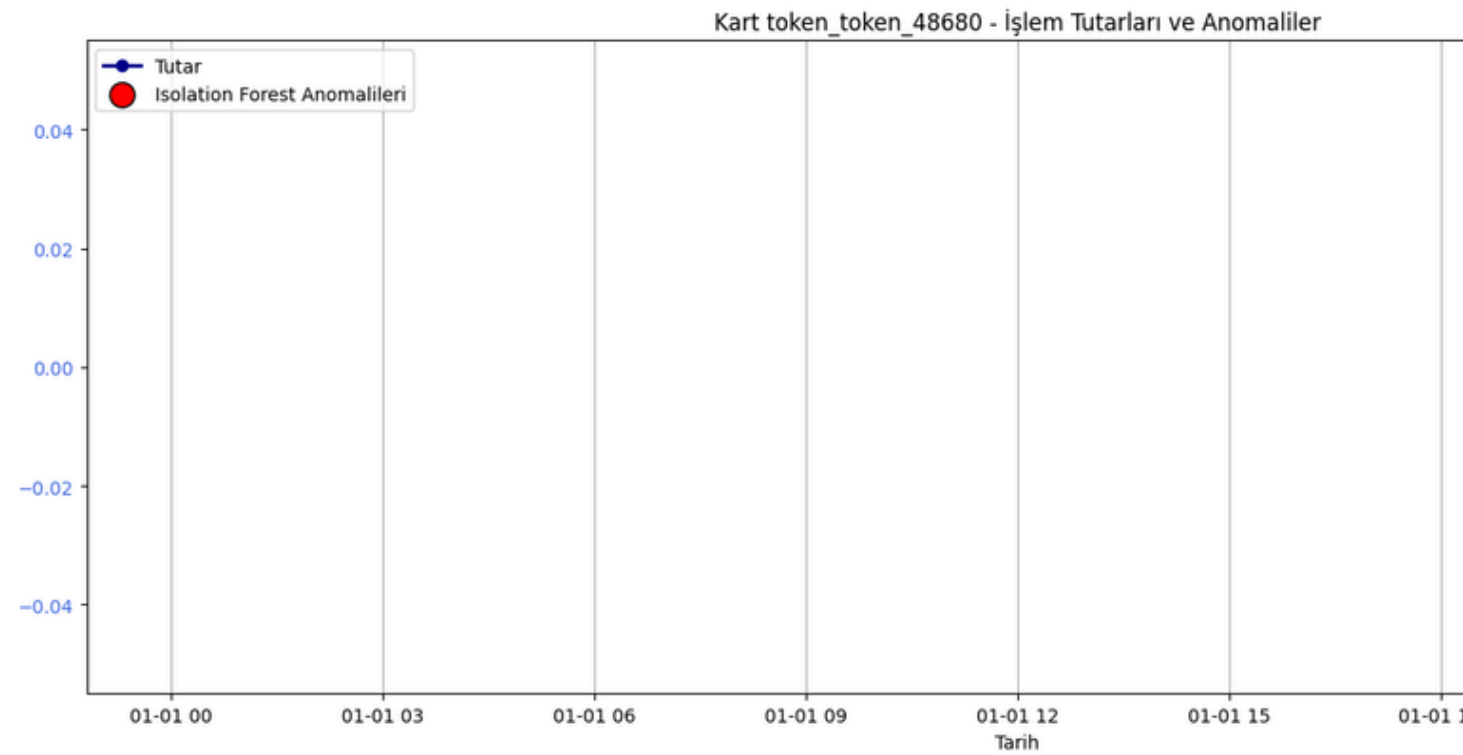
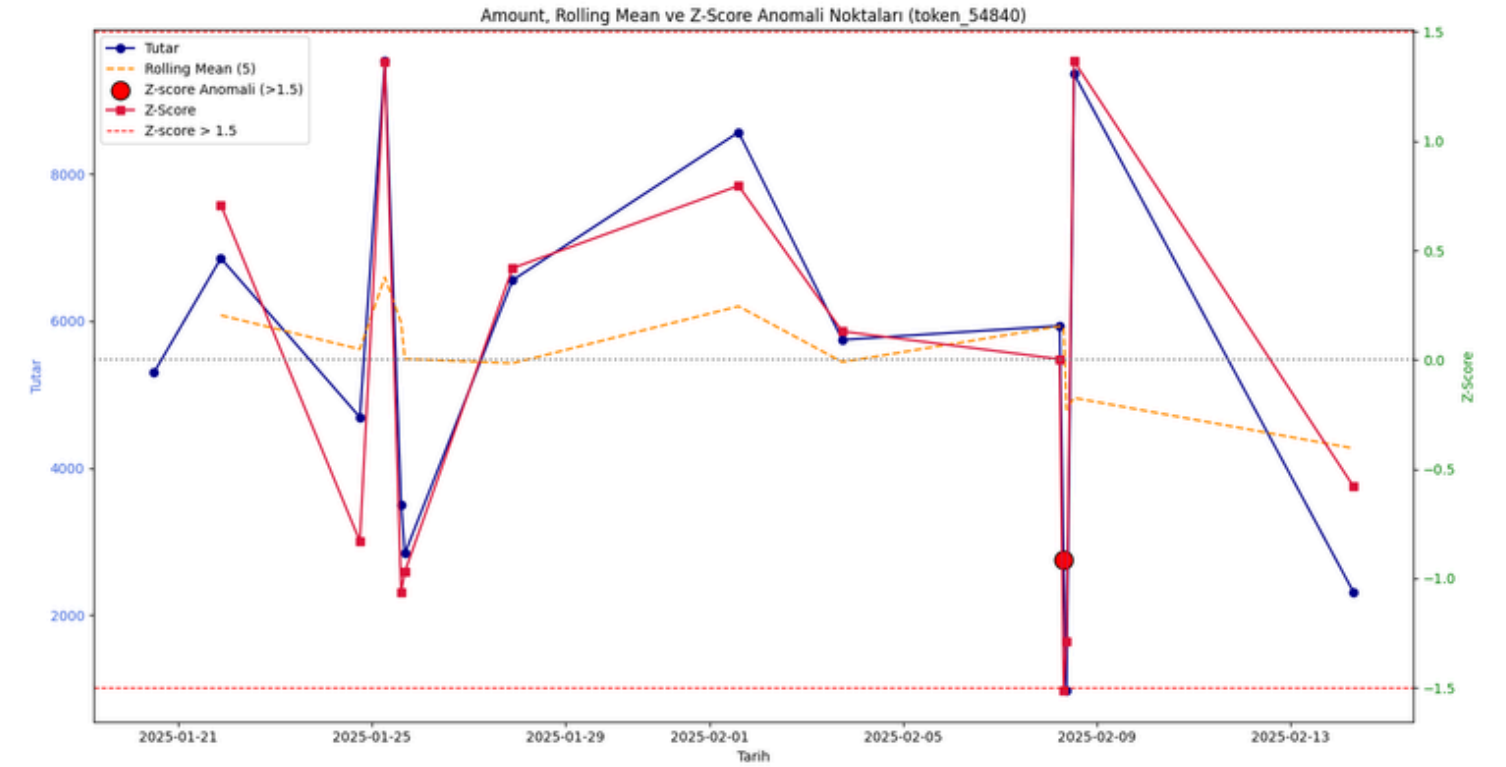
Because this method adapts to behavioral changes over time for a specific user or card, it can identify anomalous transactions even in the presence of periodic fluctuations and changing transaction patterns.



# Token\_48680



# Token\_54840



# HYPERPARAMETER OPTIMIZATION

## *Determining the Optimal Parameters Using Grid Search*

The performance of anomaly detection models depends not only on the algorithms used, but also on how those algorithms are configured. In this project, a systematic Grid Search process was applied to the window size and threshold value (k) parameters used in the Z-Score and Dynamic Thresholding methods. Multiple parameter combinations were evaluated to identify the configurations that provided the most effective anomaly detection performance.

Different parameter combinations were tested and compared based on criteria such as anomaly rate, threshold sensitivity, and suitability for the underlying time-series structure. The configurations producing the most meaningful and consistent results were then selected.

01

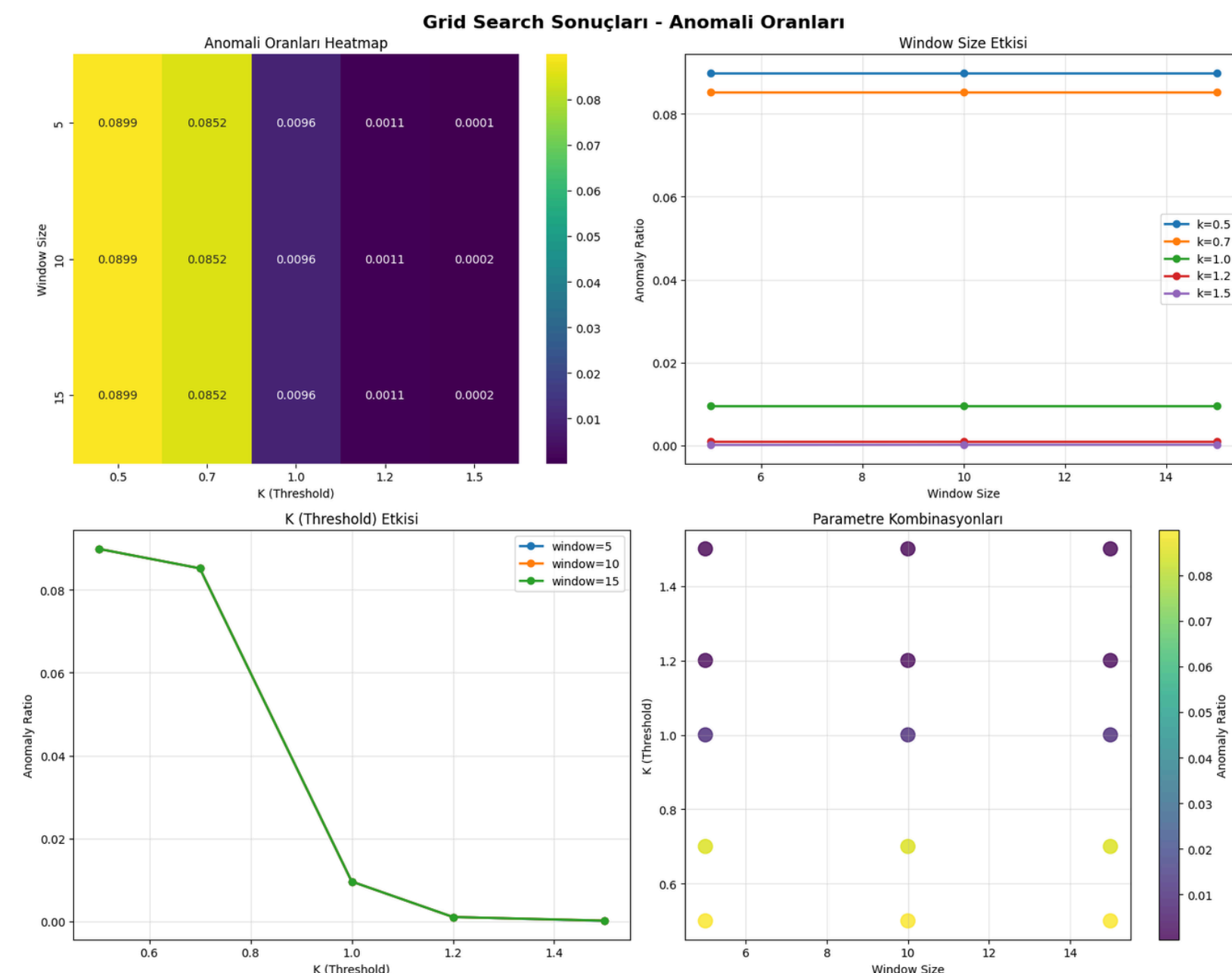
### THRESHOLD (K VALUE)

The threshold multiplier used to identify anomalies. It was applied as  $\text{mean} + k \times \text{standard deviation}$ , and 1.5 was selected as the most suitable value.

02

### WINDOW SIZE

The window length used in time-series analysis to capture and evaluate historical behavioral patterns. Values such as 5, 10, and 20 were tested.



The Z-Score method operates using a fixed threshold. The Z-score is calculated based on the mean and standard deviation within a defined window; when the score exceeds the specified threshold ( $|z| > 1.5$ ), the transaction is classified as an anomaly.

The Dynamic Thresholding method recalculates the thresholds for each window based on the underlying data. Lower and upper bounds are computed using the formula  $\text{mean} \pm k \times \text{standard deviation}$ . This allows the method to account for periodic fluctuations and detect contextual anomalies more accurately.

# LIBRARIES USED

The project used core Python libraries for data analysis, statistical processing, visualization, and machine learning implementation. These libraries supported the full anomaly detection workflow from data transformation to model evaluation and visual output generation

01

## SCIKIT-LEARN

Implementation of machine learning algorithms such as Isolation Forest and One-Class SVM.

02

## SEABORN

Advanced statistical visualization, including distribution and analysis charts.

03

## NUMPY

Numerical operations, vector/matrix manipulation, and statistical calculations.

04

## PANDAS

Data reading, cleaning, transformation, and tabular transaction analysis.

# Batuhan Meydanlı

[linkedin.com/in/batuhanmeydanli](https://www.linkedin.com/in/batuhanmeydanli)